

Policy Gradient Reinforcement Learning and Sim-to-Real Transfer for Robotic Control

Matteo Sgobba
Politecnico di Torino
Turin, Italy

s361177@studenti.polito.it

Federico Di Leo
Politecnico di Torino
Turin, Italy

s360592@studenti.polito.it

Martina Mancini
Politecnico di Torino
Turin, Italy

s362098@studenti.polito.it

Paolo Campanini
Politecnico di Torino
Turin, Italy

s360785@studenti.polito.it

Abstract

This project studies reinforcement learning for robotic control in simulation and sim-to-real transfer in a controlled sim-to-sim setting. We implemented REINFORCE and Actor-Critic variants on Hopper, then evaluated PPO, SAC, UDR, and ADR on PandaPush under source-target mass mismatch. On Hopper, the best REINFORCE baseline was $b = 20$, reaching 175.2 final return, while n -step Actor-Critic achieved the highest mean return (574.8) but with high seed variability. On PandaPush, SAC clearly outperformed PPO and reached near-perfect success. Direct source-to-target transfer achieved 98% success, while UDR and ADR recovered 100% target success under randomized mass dynamics. Source code: <https://github.com/matteosgobba/rl-sim2real-hopper>

1. Introduction

Reinforcement learning (RL) provides a framework for learning control policies through interaction with an environment, where an agent improves its behavior by maximizing cumulative reward [10]. This paradigm is attractive for robotic control because it can optimize behaviors that are difficult to program manually, especially in continuous-control tasks. However, direct real-world training is often impractical due to many interactions, unsafe exploration, repeated resets, and hardware supervision [4, 5]. Simulation enables faster and safer data collection, but policies trained in simulation may fail to transfer because simulated and real dynamics differ. This reality gap is a central challenge in sim-to-real robot learning [3]. Domain randomization addresses this issue by varying physical parameters during training, encouraging

policies to become robust to a family of possible dynamics rather than a single simulator configuration [8, 11].

This project studies these aspects in two stages. First, we implement REINFORCE and Actor-Critic on Hopper to analyze policy-gradient learning, baselines, and bootstrapping. Second, we address a simplified sim-to-real setting on PandaPush, where source and target domains differ in cube mass. We compare PPO and SAC, select the most reliable algorithm, and evaluate source-target transfer, UDR, and ADR.

Literature foundations. Policy-gradient methods optimize a parameterized stochastic policy $\pi_\theta(a | s)$ by maximizing the expected discounted return

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]. \quad (1)$$

They are well suited to continuous-control problems because actions can be sampled directly from a continuous policy distribution [10]. REINFORCE estimates the policy gradient from complete trajectories using Monte Carlo returns G_t . Although unbiased, this estimator has high variance; subtracting an action-independent baseline $b(s_t)$ reduces variance without changing the expected gradient [10]. Actor-Critic methods instead use a learned value function $V_\phi(s)$ and update the policy through a bootstrapped TD signal,

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t). \quad (2)$$

This provides more frequent and lower-variance updates than full-trajectory REINFORCE, but introduces bias through bootstrapping when the critic is inaccurate.

For the second part of the project, we considered PPO and SAC as more advanced policy-optimization algorithms. PPO improves training stability by limiting how far the updated

policy can move from the previous one. Its clipped surrogate objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (3)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio and $\hat{A}_t$ is an advantage estimate [9]. SAC is an off-policy maximum-entropy actor-critic method. Instead of maximizing only expected return, it also rewards policy entropy:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_t \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))) \right]. \quad (4)$$

The entropy term encourages exploration and can improve robustness in continuous-control tasks, which motivated its use for PandaPush [2].

The robotics literature emphasizes that real robot learning is constrained not only by sample complexity, but also by safety, reset cost, and hardware wear [4, 5]. Sim-to-real transfer addresses this by training policies in simulation and deploying them in a target environment. However, as highlighted in sim-to-real studies, transfer performance depends on how well the simulator captures the target dynamics and on whether the learned policy can tolerate modeling errors [3]. In this project, we reproduce this issue in a controlled sim-to-sim setting: source and target PandaPush differ in cube mass, so $P_{\text{source}}(s'|s, a) \neq P_{\text{target}}(s'|s, a)$.

Domain randomization provides a practical way to reduce this mismatch. Tobin et al. introduced the idea of training over randomized simulation parameters to obtain policies that generalize beyond a single simulator instance [11]. Peng et al. further showed the relevance of randomizing dynamics parameters, such as masses and physical properties, for robotic control transfer [8]. Following this idea, UDR samples parameters from a fixed uniform interval, while ADR adapts the randomization range according to boundary performance, creating a curriculum from easier to harder dynamics [6, 7].

2. Methodology

This section describes our implementation choices, training pipelines, and evaluation protocols. We implemented policy-gradient algorithms on Hopper, then used Stable-Baselines3 for source-target transfer and domain randomization on PandaPush.

2.1. Hopper Policy Gradient Setup

2.1.1. Environment and Evaluation Protocol

The first experiments were conducted on `Hopper-v4`, a MuJoCo continuous-control locomotion task where a one-legged robot must move forward while maintaining balance. We first inspected the environment structure required by the

assignment. The observation space is a continuous `Box` with shape `(11, )`, containing joint positions and velocities. The action space is a continuous `Box` with shape `(3, )`, bounded in `[-1, 1]`, corresponding to actuator commands for the thigh, leg, and foot joints. This is consistent with the Gaussian policies used by the implemented agents.

We also extracted the body masses from the MuJoCo model using `env.unwrapped.model.body_names` and `env.unwrapped.model.body_mass`. The model contains five bodies, six degrees of freedom, and three actuators. The extracted masses were: world 0.0000 kg, torso 3.6652 kg, thigh 4.0579 kg, leg 2.7814 kg, and foot 5.3156 kg. This preliminary inspection was used to verify the state/action structure and physical parameters before training.

After this inspection, we trained the implemented policy-gradient agents using seeds 0, 1, 2. Performance was assessed through periodic evaluation return, final average return, standard deviation across seeds, and training time. Evaluation returns were computed without parameter updates, while final average return summarized end-of-training performance. Training time was included as an auxiliary metric, but it was interpreted together with return because weak Hopper policies may terminate episodes earlier by falling and therefore require less simulation time.

2.1.2. Implemented Algorithms

For REINFORCE, we implemented a stochastic Gaussian policy with two hidden layers and nonlinear activations, consistently with the policy-gradient objective in Eq. (1). At each time step, the policy sampled a continuous action and stored its log-probability. After the end of the episode, discounted returns were computed as

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k, \quad (5)$$

and the policy was updated using the Monte Carlo policy-gradient loss

$$\mathcal{L}_\pi = - \sum_{t=0}^T \log \pi_\theta(a_t|s_t) (G_t - b), \quad (6)$$

which corresponds to minimizing the negative sample estimate of the policy-gradient objective. Here, b is an optional constant baseline used to shift the return estimate without depending on the sampled action. We tested REINFORCE both without a baseline and with several constant baseline values, $b \in \{0, 20, 50, 100\}$. This was included as a baseline-sensitivity analysis to evaluate whether a simple constant baseline could reduce variance and improve learning stability.

For Actor-Critic, we used two separate neural networks: an actor representing the Gaussian policy and a critic estimating the state value $V_\phi(s)$. Differently from REINFORCE,

the actor was not updated from full Monte Carlo returns only, but from a bootstrapped temporal-difference signal related to Eq. (2). The one-step version used the temporal-difference target

$$y_t = r_t + \gamma V_\phi(s_{t+1})(1 - d_t), \quad (7)$$

where d_t indicates episode termination. The advantage estimate was computed as

$$A_t = y_t - V_\phi(s_t), \quad (8)$$

and used to update the actor, while the critic was trained by minimizing the mean squared error between $V_\phi(s_t)$ and y_t . We also implemented an n -step variant with $n = 10$, in order to reduce the dependence on a purely one-step bootstrapped target and provide a less myopic learning signal. Entropy regularization and gradient clipping were used to improve numerical stability.

2.2. PandaPush Transfer Setup

2.2.1. Source and Target Domains

The second part of the project was conducted on PandaPush-v3, a goal-conditioned robotic manipulation task from panda-gym. The agent controls a Franka Panda robot and must push a cube toward a target position on the table. The observation is dictionary-based and includes the robot/object state, the achieved goal, and the desired goal. The action space is continuous and controls the end-effector displacement.

The assignment defines a sim-to-sim transfer setting that mimics the sim-to-real problem. The source and target environments have the same task structure but differ in the cube mass:

$$m_{\text{source}} = 1.0 \text{ kg}, \quad m_{\text{target}} = 5.0 \text{ kg}. \quad (9)$$

This mass mismatch changes the transition dynamics and creates a controlled reality gap. We considered three main training/evaluation configurations:

- **source** $\rightarrow$ **source**: training and testing on the source domain, used as a sanity check;
- **source** $\rightarrow$ **target**: training on the source domain and testing on the target domain, used to measure transfer degradation;
- **target** $\rightarrow$ **target**: training and testing on the target domain, used as an upper-bound reference.

Each trained policy was evaluated over 50 episodes without further learning. For each training–test configuration, we reported mean return, return standard deviation, and success rate. The mean return measures the average dense reward accumulated during evaluation, the standard deviation captures episode-level variability, and the success rate measures the fraction of episodes in which the object reached the desired goal. Since the reward is dense, return also captures differences in trajectory efficiency even when the success rate saturates.

2.2.2. PPO/SAC and Domain Randomization Pipeline

For PandaPush, we used Stable-Baselines3 to train PPO and SAC. PPO was considered as an on-policy policy-gradient baseline and optimizes the clipped surrogate objective introduced in Eq. (3), which constrains policy updates by clipping the probability ratio between the new and old policies. SAC was considered as an off-policy maximum-entropy actor-critic method and follows the entropy-regularized objective in Eq. (4), balancing expected return with policy entropy. Both algorithms were trained and monitored through TensorBoard logs, including rollout success rate, mean episode reward, and episode length. PandaPush models were trained with a fixed training seed, set to 0. To reduce the dependence of the reported metrics on stochastic evaluation episodes, each trained model was evaluated over 50 episodes using evaluation seeds 0, 1, and 2. This comparison was used to select the algorithm for the subsequent transfer and domain randomization experiments.

For PPO, we explored different learning rates and entropy coefficients, and also included longer exploratory runs up to 5M timesteps to test whether additional training budget improved the learning behavior. PPO was then retained as the on-policy baseline for the final comparison. For SAC, we kept the Stable-Baselines3 default parameters except for the explicitly tuned learning rate and entropy coefficient. We tested both automatic entropy tuning and a fixed entropy coefficient of 0.01. Since fixed-domain SAC reached saturated performance with both entropy settings, we used automatic entropy tuning for domain randomization to avoid manually fixing the exploration weight under varying mass dynamics.

Uniform Domain Randomization was implemented through an environment wrapper that samples the cube mass at every environment reset:

$$m \sim \mathcal{U}(m_{\min}, m_{\max}). \quad (10)$$

The policy was trained on the randomized source environment and evaluated on both source and target domains. The UDR range was set to $[1.0, 6.0]$, which includes the source mass and extends beyond the target mass without assuming knowledge of the target dynamics. We used a single UDR range because UDR requires a manually fixed interval; therefore, we selected one reasonable non-target-specific range rather than tuning several intervals around the target mass.

Automatic Domain Randomization used the same wrapper structure, but the mass interval was adapted during training. ADR started from the source-like interval $[1.0, 1.5]$ and expanded or contracted its boundaries according to the agent’s success at the current range limits. We used an ADR step of 0.25 and a boundary sampling probability of 0.5. Unlike UDR, ADR explicitly depends on the allowed expansion limit, so we repeated it with maximum mass limits equal to 6.0, 8.0, and 10.0 to evaluate the robustness of the adaptive curriculum.

Table 1. PandaPush experimental setup.

Experiment	Algorithm	Train domain	Mass setting	Steps	Eval eps.
PPO baseline	PPO	source/target	fixed	1M / 5M	50
SAC baseline	SAC	source/target	fixed 1.0/5.0 kg	1M	50
UDR	SAC	source	$m \sim \mathcal{U}(1, 6)$	1M	50
ADR	SAC	source	adaptive, max 6/8/10	1M	50

Table 2. Main PandaPush hyperparameters.

Algorithm	Learning rate	Entropy coef.	Reward
PPO	$4 \cdot 10^{-4} / 4 \cdot 10^{-3}$	0 / 0.01	dense
SAC	$4 \cdot 10^{-4}$	auto / 0.01	dense
SAC + UDR	$4 \cdot 10^{-4}$	auto	dense
SAC + ADR	$4 \cdot 10^{-4}$	auto	dense

3. Experimental Results

3.1. Hopper: REINFORCE and Actor-Critic

Fig. 1 reports the evaluation return during training on Hopper. REINFORCE without baseline learned only weakly, while $b = 20$ gave the best REINFORCE performance. Larger baselines did not help: $b = 50$ increased variability and $b = 100$ strongly degraded learning. Actor-Critic one-step remained weak, whereas the n -step variant achieved the best mean return but with high variability across seeds.

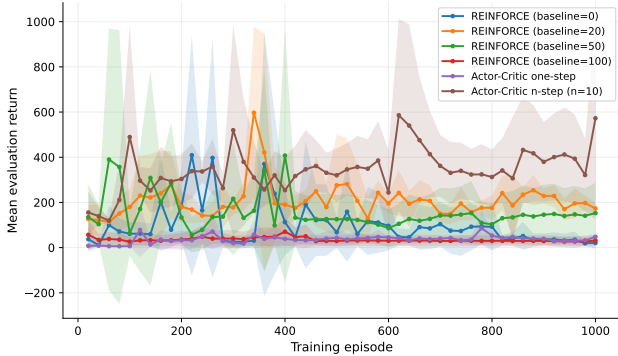


Figure 1. Evaluation return on Hopper-v4. Curves show the mean over seeds 0, 1, 2, with one standard deviation.

Table 3 gives the quantitative breakdown for the final Hopper evaluation across seeds 0, 1, 2. The n -step Actor-Critic achieved the highest mean final return, 574.8, but also the largest standard deviation, 446.6. Among REINFORCE variants, $b = 20$ gave the best mean return, while $b = 100$ performed close to the failed baseline case. Training time should not be interpreted alone, since weak policies often fall earlier and therefore require less simulation time.

3.2. PandaPush: PPO, SAC, and Transfer

Fig. 2 compares the PPO baseline and the best fixed-domain SAC runs on PandaPush over 1M timesteps. PPO used learning rate $4 \cdot 10^{-4}$ and entropy coefficient 0, while SAC used learning rate $4 \cdot 10^{-4}$ and fixed entropy coefficient 0.01, which was marginally better than automatic entropy tuning in the fixed-domain setting. Across the tested PPO configu-

Table 3. Final Hopper results averaged across seeds 0, 1, 2.

Method	Final return	Std.	Time [s]
REINFORCE, $b = 0$	20.8	26.1	15.2
REINFORCE, $b = 20$	175.2	21.6	29.2
REINFORCE, $b = 50$	155.9	139.2	16.9
REINFORCE, $b = 100$	31.8	26.5	4.7
Actor-Critic one-step	48.2	11.1	15.5
Actor-Critic n -step	574.8	446.6	43.1

rations, including exploratory runs up to 5M timesteps, PPO always remained near a low success-rate plateau, while SAC rapidly reached near-perfect success. SAC was therefore selected for transfer and domain randomization.

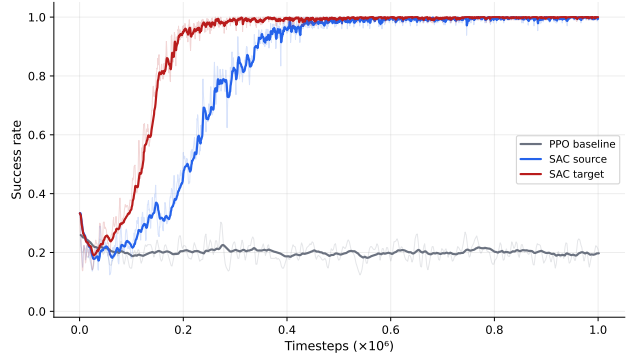


Figure 2. Rollout success rate of the PPO baseline and fixed-domain SAC runs on PandaPush.

Table 4 reports the source-target configurations using the selected fixed-domain SAC setup, with learning rate $4 \cdot 10^{-4}$ and fixed entropy coefficient 0.01. Source $\rightarrow$ source reached 100% success, direct source $\rightarrow$ target reached 98%, and target $\rightarrow$ target reached 100%. The lower return and larger standard deviation in source $\rightarrow$ target indicate reduced efficiency and consistency under mass mismatch.

Table 4. SAC source-target evaluation results on PandaPush averaged across evaluation seeds 0, 1, 2.

Train $\rightarrow$ Test	Mean return	Std.	Success
Source $\rightarrow$ Source	-0.399	0.283	100%
Source $\rightarrow$ Target	-0.510	0.569	98%
Target $\rightarrow$ Target	-0.403	0.294	100%

3.3. Domain Randomization

Fig. 3 compares fixed-source SAC with UDR and ADR using the same SAC learning rate, $4 \cdot 10^{-4}$, and automatic entropy tuning, following the entropy-tuning rationale described in Sec. 2.2.2. Both randomization strategies converged successfully and reached 100% target success rate.

Table 5 summarizes final target-domain evaluation for SAC policies trained on the fixed source domain, the fixed target domain, and randomized source domains with UDR

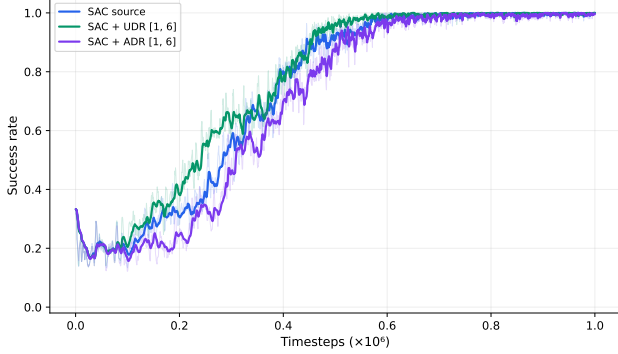


Figure 3. Rollout success rate of SAC with and without domain randomization.

or ADR. All policies in this comparison use $lr = 4 \cdot 10^{-4}$ and automatic entropy tuning, consistently with the domain-randomization setting. UDR and ADR removed the residual success-rate gap of direct source $\rightarrow$ target transfer. Dense returns remained high, although the broadest ADR range slightly reduced efficiency and increased variability.

Table 5. Final target-domain SAC evaluation averaged across evaluation seeds.

Method	Train env.	Mass setting	Target return	Success
SAC source	source	fixed 1 kg	-0.544 ± 0.691	98%
SAC target	target	fixed 5 kg	-0.447 ± 0.336	100%
SAC + UDR	source	$\mathcal{U}(1, 6)$	-0.437 ± 0.332	100%
SAC + ADR	source	adaptive, max 6	-0.437 ± 0.307	100%
SAC + ADR	source	adaptive, max 8	-0.439 ± 0.327	100%
SAC + ADR	source	adaptive, max 10	-0.533 ± 0.412	100%

4. Discussion

The Hopper results show that simple policy-gradient methods are highly sensitive to the quality and scale of the learning signal. In REINFORCE, $b = 20$ was close enough to the empirical return scale to reduce update variance without changing the expected policy-gradient direction. Larger baselines degraded learning: $b = 50$ increased variability, while $b = 100$ shifted many early advantages $G_t - b$ toward negative or poorly scaled values. Thus, a constant baseline should be chosen according to the observed reward scale, not arbitrarily.

The Actor-Critic results highlight the trade-off introduced by bootstrapping. The one-step version should in principle update more frequently than REINFORCE, but it relied on an inaccurate early critic, making the TD advantage noisy and biased. The n -step variant used a less myopic signal by incorporating more observed rewards before bootstrapping. This improved mean performance, but the large standard deviation shows that the method remained sensitive to initialization and stochastic training effects. Therefore, n -step updates improved credit assignment, but did not fully solve the stability limits of the implemented agents.

In PandaPush, SAC was more reliable than PPO and was selected for transfer. This result is consistent with the stronger stability of the off-policy maximum-entropy pipeline in our setting, especially under continuous actions and dense rewards. Direct source $\rightarrow$ target evaluation confirmed the expected dynamics mismatch: the same action sequence affects a 1.0 kg cube and a 5.0 kg cube differently. Target $\rightarrow$ target is an upper-bound reference, but in real sim-to-real settings it would require many physical interactions, costly resets, and unsafe exploration, making it impractical.

UDR and ADR recovered robust target performance by training over randomized mass dynamics. The UDR range $[1, 6]$ includes the source mass and extends beyond the target mass without being narrowly centered around it, avoiding a target-specific shortcut. ADR started from $[1, 1.5]$ and expanded the interval as a curriculum, first exposing the policy to easier source-like dynamics and then to harder configurations. However, UDR depends on a manually chosen range, while ADR introduces additional design choices such as boundary sampling, expansion rules, and success thresholds. Wider randomization did not prevent convergence, but the broadest ADR setting reduced return efficiency.

The main limitation is that the study is sim-to-sim rather than real sim-to-real. Only cube mass was randomized, while real robots also involve friction, delays, sensing noise, actuator uncertainty, and unmodeled contacts. Moreover, the number of training seeds was limited by computational cost. Since most SAC-based policies saturated success rate, dense return and variability were more informative than success rate alone. Overall, the results support the usefulness of domain randomization, but also show that robustness depends on the chosen randomization design and on the stability of the underlying reinforcement learning algorithm.

5. Conclusion

This project investigated RL for continuous-control robotic tasks from two perspectives. On Hopper, REINFORCE was sensitive to the baseline magnitude, while n -step Actor-Critic achieved the highest mean return but remained variable across seeds. On PandaPush, SAC clearly outperformed PPO and was therefore used for transfer and domain randomization. Direct source-to-target transfer achieved strong performance, while UDR and ADR closed the residual success-rate gap under randomized mass dynamics.

Overall, the results confirm that domain randomization can improve robustness when target dynamics are unavailable during training, but it also introduces design choices such as the UDR mass range and ADR adaptation rule. Future work could consider richer mismatch sources, including friction, sensing noise, delays, and actuator uncertainty, and compare SAC with alternative off-policy methods or improved critic formulations such as DDPG/TD3-style approaches [1].

Acknowledgments

The authors declare that Generative AI tools were not used for writing this project report. All textual content, experimental interpretation, result discussion, and final conclusions were written and reviewed by the group members.

Generative AI tools were used only as coding support during the development of the submitted project code. The system used was ChatGPT, GPT-5.5, published by OpenAI. Its use was limited to accelerating auxiliary and selected implementation tasks under continuous human supervision. In particular, it supported the preparation of the software setup and installation requirements, as well as the development of scripts used to aggregate experimental outputs and generate the plots for both parts of the project, including the processing of .pt files and TensorBoard logs. It also supported the implementation or refinement of selected code functionalities introduced during development, such as the n -step critic and domain randomization components. In addition, it assisted with refactoring, debugging, and consistency checks in selected scripts, including issues arising from overlapping contributions by multiple group members.

All generated suggestions were manually inspected, tested, and adapted by the authors. The design choices, algorithmic decisions, experimental protocol, evaluation procedure, interpretation of results, and final judgment on the correctness of the implementation remained entirely under the responsibility of the group members.

References

- [1] Quentin Gallouédec, Nicolas Cazin, Emmanuel Dellandréa, and Liming Chen. panda-gym: Open-source goal-conditioned environments for robotic learning, 2021. [5](#)
- [2] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018. [2](#)
- [3] Sebastian Höfer, Kostas Bekris, Ankur Handa, Juan Camilo Gamboa, Florian Golemo, Melissa Mozifian, Chris Atkeson, Dieter Fox, Ken Goldberg, John Leonard, C. Karen Liu, Jan Peters, Shuran Song, Peter Welinder, and Martha White. Perspectives on sim2real transfer for robotics: A summary of the r:ss 2020 workshop, 2020. [1](#), [2](#)
- [4] Jens Kober, J. Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32:1238–1274, 2013. [1](#), [2](#)
- [5] Petar Kormushev, Sylvain Calinon, and Darwin Caldwell. Reinforcement learning in robotics: Applications and real-world challenges. *Robotics*, 2:122–148, 2013. [1](#), [2](#)
- [6] Fabio Muratore, Fabio Ramos, Greg Turk, Wenhao Yu, Michael Gienger, and Jan Peters. Robot learning from randomized simulations: A review, 2022. [2](#)
- [7] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lil-
ian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving rubik’s cube with a robot hand, 2019. [2](#)
- [8] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, page 3803–3810. IEEE, 2018. [1](#), [2](#)
- [9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. [2](#)
- [10] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018. [1](#)
- [11] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world, 2017. [1](#), [2](#)